Django Backend Architecture

1️⃣ Project root (backend root)

```
.
├── manage.py
├── db.sqlite3
├── requirements.txt
├── README.md
├── realestate_backend/
├── authapp/
├── users/
├── listings/
└── bookings/
```

🔹 manage.py

Command-line entry point for Django.

Used to:

- Run the server: python manage.py runserver
- Apply migrations
- Create superusers
- Run tests

Internally:

- Loads settings.py
- Bootstraps the Django app

🔹 db.sqlite3

Your **development database**.

- SQLite is default for local dev
- Stores:
    - Users
    - Listings
    - Bookings
    - Auth data

In production → usually PostgreSQL.

🔹 **requirements.txt**

Python dependencies list.

Example contents:

Django

djangorestframework

djangorestframework-simplejwt

Used for:

pip install -r requirements.txt

🔹 **README.md**

Documentation for:

- Setup instructions
- API usage
- Project overview

Important for team & deployment.

2️⃣ Core Django project: realestate_backend/

This is the **configuration & wiring layer** (not business logic).

```
realestate_backend/
 ├── settings.py
 ├── urls.py
 ├── asgi.py
 ├── wsgi.py
└── __init__.py
```

🔹 settings.py

Central configuration hub.

Controls:

- Installed apps (authapp, users, listings, bookings)
- Database connection
- Middleware
- Authentication
- REST Framework settings
- Security settings

Example responsibilities:

```
INSTALLED_APPS = [
  'rest_framework',
  'users',
  'listings',
  'bookings',
]
```

🔹 urls.py

Global URL router.

Connects app URLs together:

```
urlpatterns = [
    path("api/auth/", include("authapp.urls")),
    path("api/users/", include("users.urls")),
    path("api/listings/", include("listings.urls")),
    path("api/bookings/", include("bookings.urls")),
]
```

Acts like a **traffic dispatcher**.

🔹 **wsgi.py**

Used for **production servers** (Gunicorn, uWSGI).

- Handles synchronous HTTP requests
- Required for deployment

🔹 **asgi.py**

Used for **async features**:

- WebSockets
- Real-time notifications
- Chat
- Live booking updates

Useful later for:

- Booking status updates
- Agent chat

3️⃣ Apps = Domain-based backend modules

Each app represents a **business domain**.

🔐 authapp/ – Authentication & tokens

Purpose:

Handles login, registration, JWT tokens, authentication logic

```
authapp/
├── models.py
├── serializers.py
├── views.py
├── urls.py
├── apps.py
```

🔹 models.py

Defines **auth-related models** (if any).

Examples:

- OTP codes
- Refresh tokens
- Password reset tokens

Often light if using Django's built-in User.

🔹 serializers.py

Handles:

- Login input validation
- Token serialization
- Password validation

Example:

```
class LoginSerializer(serializers.Serializer):
```

```
email = serializers.EmailField()
password = serializers.CharField()
```

🔹 **views.py**

Auth logic:

- Login
- Register
- Token refresh
- Logout

Example:

POST /api/auth/login/

POST /api/auth/register/

🔹 **urls.py**

Routes auth endpoints:

/api/auth/login/

/api/auth/register/

/api/auth/refresh/

🔹 **apps.py**

App configuration metadata.

👤 **users/ – User profiles & roles**

Purpose:

Manages user data beyond authentication

users/

```
├── models.py
├── serializers.py
├── views.py
├── urls.py
```

🔹 **models.py**

User-related domain data:

- Profile
- Role (buyer, agent, landlord, admin)
- Phone number
- Verification status

Example:

```
class Profile(models.Model):
  user = models.OneToOneField(User)
  role = models.CharField(...)
```

🔹 **serializers.py**

Converts user models ↔ JSON.

Handles:

- Profile updates
- Role exposure
- Read/write separation

🔹 **views.py**

User APIs:

- Get profile

- Update profile
- List agents
- View public user info

🔹 **urls.py**

Endpoints:

/api/users/me/

/api/users/<id>/

/api/users/agents/

🏠 **listings/ – Properties & real estate inventory**

Purpose:

Manages properties available for booking

listings/

```
├── models.py
├── serializers.py
├── views.py
├── urls.py
```

🔹 **models.py**

Defines property data:

Typical fields:

- Title
- Description
- Price

- Location
- Availability
- Owner/agent

Example:

```
class Listing(models.Model):
  owner = models.ForeignKey(User)
  price_per_night = models.DecimalField(...)
```

🔹 **serializers.py**

Controls:

- Public listing data
- Owner-only fields
- Validation

Example:

- Price must be positive
- Availability dates valid

🔹 **views.py**

Listing APIs:

- Create listing
- Search listings
- Filter by price/location
- View listing details

🔹 **urls.py**

Endpoints:

/api/listings/

/api/listings/<id>/

/api/listings/search/

📅 bookings/ – Reservations & availability

Purpose:

Handles booking logic & business rules

```
bookings/
├── models.py
├── serializers.py
├── views.py
├── urls.py
```

🔹 models.py

Defines booking data:

Fields:

- User
- Listing
- Check-in / check-out
- Status (pending, confirmed, canceled)
- Total price

Example:

```
class Booking(models.Model):
  listing = models.ForeignKey(Listing)
  user = models.ForeignKey(User)
  status = models.CharField(...)
```

🔹 **serializers.py**

Validates:

- Date conflicts
- Availability
- Price calculation

This is where **business rules live**.

🔹 **views.py**

Booking workflows:

- Create booking
- Cancel booking
- View user bookings
- Owner booking management

🔹 **urls.py**

Endpoints:

/api/bookings/

/api/bookings/my/

/api/bookings/<id>/cancel/

4️⃣ **__pycache__/ folders**

These contain:

- Compiled Python bytecode (.pyc)
- Auto-generated
- Safe to ignore

- Never commit to git

5️⃣ How everything works together (real flow)

🔁 Booking a property

1. **User logs in**
   → authapp
2. **User profile fetched**
   → users
3. **User searches listings**
   → listings
4. **User books a property**
   → bookings
5. **Booking validated**
   → checks conflicts, price, ownership
6. **Database updated**
7. **Response returned as JSON**

🔁 High-level request flow (API backend

```
Client (Web / Mobile / Frontend)
        |
        |  HTTP Request (JSON)
        v
┌──────────────────────────┐
│ Django ASGI / WSGI Server│
│ (runserver / gunicorn)   │
└──────────┬───────────────┘
           |
           v
┌──────────────────────────┐
│ Middleware Stack         │
│ - Auth                   │
│ - CORS                   │
│ - Security               │
│ - Logging                │
└──────────┬───────────────┘
           |
           v
┌──────────────────────────┐
│ URL Dispatcher           │
│ realestate_backend/urls  │
└──────────┬───────────────┘
           |
           v
┌──────────────────────────┐
│ App URLs                 │
│ auth / users / listings  │
│ bookings                 │
└──────────┬───────────────┘
           |
```

```
              v
┌──────────────────────────┐
│ View (APIView / ViewSet) │
│ - Permissions            │
│ - Business Logic         │
└──────────────────────────┘
              |
              |
              v
┌──────────────────────────┐
│ Serializer               │
│ - Validation             │
│ - Data transformation    │
└──────────────────────────┘
              |
              |
              v
┌──────────────────────────┐
│ ORM / Models             │
│ - Queries                │
│ - Transactions           │
└──────────────────────────┘
              |
              |
              v
┌──────────────────────────┐
│ Database (SQLite / PG)   │
└──────────────────────────┘
              |
              |
              v
┌──────────────────────────┐
│ JSON Response            │
│ status + data            │
└──────────────────────────┘
              |
              |
              v
Client
```

🏠 Example: Booking a property (concrete flow)

POST /api/bookings/

Authorization: Bearer <JWT>

```
Client
  |
  v
ASGI / WSGI
  |
  v
Middleware
  ├─ JWT Authentication
  ├─ User attached to request
  |
  v
URLConf
  └─ bookings/urls.py
        |
        v
BookingCreateView
  ├─ permission_classes = IsAuthenticated
  |
  v
BookingSerializer
  ├─ validate dates
  ├─ check availability
  ├─ calculate price
  |
  v
Booking Model
  ├─ create booking
  |
  v
Database
  |
  v
Serialized Booking JSON
  |
  v
HTTP 201 Created
```

🔐 Auth request flow (JWT login)

POST /api/auth/login/

```
Client
  |
  v
Auth View
  ├─ validate credentials
  ├─ generate JWT
  |
  v
Token Serializer
  |
  v
JSON Response
  ├─ access token
  ├─ refresh token
```

React Frontend Architecture

1️⃣ High-level architecture (mental model)

Your frontend follows a **clean layered architecture**:

Pages (Screens)

↓

Components (UI building blocks)

↓

Hooks & Contexts (State & logic)

↓

Services (API calls)

↓

Django REST Backend

This separation is **exactly what you want** for scalability.

2️⃣ Root level (build & tooling)

.

├── index.html

```
├── vite.config.js
├── package.json
├── eslint.config.js
├── public/
└── src/
```

🔹 **index.html**

The **single HTML entry point** (SPA).

Contains:

React mounts here.

🔹 **vite.config.js**

Vite build configuration:

- Fast dev server
- Hot module reload
- Optimized production builds

🔹 **package.json**

Defines:

- Dependencies (React, Router, Axios, etc.)
- Scripts:
- "dev", "build", "preview", "test"

🔹 **eslint.config.js**

Code quality rules:

- Prevents bugs
- Enforces style consistency

🔹 **public/**

Static assets served **as-is**:

- Favicons
- SVGs
- Images

3️⃣ src/ — the real application

```
src/
├── main.jsx
├── App.jsx
├── routes/
├── pages/
├── components/
├── contexts/
├── hooks/
├── services/
├── utils/
├── assets/
└── tests/
```

🔁 App bootstrap flow

🔹 **main.jsx**

Application entry point.

Responsibilities:

- Create React root
- Wrap app in providers (Auth, Router)

Typical flow:

```
<AuthProvider>
  <BrowserRouter>
    <App />
  </BrowserRouter>
</AuthProvider>
```

🔹 **App.jsx**

Top-level app shell.

Usually contains:

- Global layout
- Route mounting

Think of it as:

“The container for everything”

4️⃣ **Routing layer**

src/routes/AppRoutes.jsx

🔹 **AppRoutes.jsx**

Defines **URL → Page mapping**.

Example:

```
<Route path="/login" element={<Login />} />
<Route path="/property/:id" element={<PropertyDetails />} />
```

Handles:

- Public routes
- Protected routes (auth required)
- Dashboard routes

5️⃣ Pages (screens)

src/pages/

Each file = **one screen / route**.

Examples:

- Home.jsx
- PropertyDetails.jsx
- Dashboard.jsx
- AddProperty.jsx

🔹 _PageShell.jsx

Shared page wrapper:

- SEO
- Layout consistency
- Meta tags

🔹 Page responsibilities

Pages:

- Fetch data
- Coordinate components
- Handle navigation

They **do NOT**:

- Call APIs directly

- Store global state

That’s done via services & hooks 👇

6️⃣ Components (reusable UI)

src/components/

├── common/

└── layout/

🔹 **layout/**

Global UI:

- Header.jsx
- Footer.jsx
- MainLayout.jsx

Used across pages.

🔹 **common/**

Reusable, generic components:

- SEO.jsx
- Buttons
- Modals (future)

7️⃣ State management (Auth & shared state)

src/contexts/

├── AuthContext.jsx

🔹 **AuthContext.jsx**

Central authentication state:

- Current user
- JWT tokens
- Login/logout logic

Provides:

{ user, login, logout, isAuthenticated }

🔹 **Why this matters**

Avoids:

- Prop drilling
- Duplicate auth logic
- Token mismatch bugs

8️⃣ **Custom hooks (logic reuse)**

src/hooks/

```
├── useAuth.js
├── useFetch.js
```

🔹 **useAuth.js**

Thin wrapper around AuthContext.

Usage:

```
const { user, logout } = useAuth();
```

🔹 **useFetch.js**

Reusable data fetching logic:

- Loading state
- Error handling

- API calls

Prevents copy-pasting fetch logic.

9️⃣ Services (API communication layer)

src/services/

This is the **bridge to Django**.

🔹 apiClient.js

Central HTTP client (Axios/fetch):

- Base URL
- Authorization headers
- Interceptors

Example:

Authorization: Bearer <JWT>

🔹 endpoints.js

Single source of truth for API URLs:

export const BOOKINGS = "/api/bookings/";

Avoids hard-coding strings everywhere.

🔹 Domain services

Each matches a Django app:

Service	Django App
auth.service.js	authapp
users.service.js	users
listings.service.js	listings

Service	Django App
bookings.service.js	bookings

Example:

createBooking(data)

→ POST /api/bookings/

🔁 Frontend → Backend request flow (real example)

User books a property

```
PropertyDetails.jsx
  |
  v
bookings.service.js
  |
  v
apiClient.js
  |
  v
POST /api/bookings/
  |
  v
Django bookings app
```

🔟 Utilities & helpers

src/utils/

- **constants.js**

Global constants:

- Roles
- Status codes
- Config values

🔹 **format.js**

Formatting helpers:

- Dates
- Prices
- Text

🔹 **storage.js**

Local/session storage abstraction:

- JWT persistence
- User caching

1️⃣1️⃣ **Assets & styling**

src/assets/

Includes:

- CSS
- Images
- Icons

Supports:

- Global styles
- Component-level styles

1️⃣ 2️⃣ Testing

src/tests/

🔹 **smoke.test.jsx**

Basic sanity test:

- App renders
- No crash on load

Foundation for:

- Component tests
- Integration tests

🧭 A. High-level Frontend Request Flow

```
(START)
  |
  v
[User Interaction]
(click / submit / navigation)
  |
  v
[Page Component]
(Home, PropertyDetails, Dashboard)
  |
  v
[UI Components]
(Header, Forms, Cards)
  |
  v
```

```
[Custom Hook]
(useAuth / useFetch)
  |
  v
[Service Layer]
(auth.service / listings.service / bookings.service)
  |
  v
[API Client]
(apiClient.js)
  |
  v
[HTTP Request]
(REST / JSON)
  |
  v
[Django Backend API]
  |
  v
[HTTP Response]
(JSON)
  |
  v
[Service Layer]
(parse response)
  |
```

```
  v
[Hook Updates State]
(loading / data / error)
  |
  v
[Context Updates Global State]
(AuthContext)
  |
  v
[React Re-render]
  |
  v
(UI Updated)
  |
  v
(END)
```

🔐 B. Authentication Flow (Login)

```
(START)
  |
  v
[User submits Login Form]
  |
  v
<Login Page>
  |
```

```
  v
useAuth()
  |
  v
auth.service.login()
  |
  v
apiClient.post("/api/auth/login/")
  |
  v
{Login Successful?}
  |         |
 No        Yes
  |         |
  v         v
[Show Error] [Receive JWT Tokens]
              |
              v
         storage.js (save tokens)
              |
              v
         AuthContext.setUser()
              |
              v
        Protected Routes Unlocked
              |
```

```
          v
        (END)
```

🏠 C. Listings Retrieval Flow

```
(START)
  |
  v
[User visits Listings Page]
  |
  v
<PropertyGrid Page>
  |
  v
useFetch()
  |
  v
listings.service.getListings()
  |
  v
apiClient.get("/api/listings/")
  |
  v
[JSON Response]
  |
  v
useFetch updates state
```

```
  |
  v
<Property Cards Rendered>
  |
  v
(END)
```

📅 D. Booking Creation Flow

```
(START)
  |
  v
[User clicks "Book Now"]
  |
  v
<PropertyDetails Page>
  |
  v
useAuth() → check login
  |
  v
{Authenticated?}
  |        |
 No       Yes
  |        |
  v        v
[Redirect]  bookings.service.createBooking()
```

```
        |
        v
  apiClient.post("/api/bookings/")
        |
        v
   {Booking Accepted?}
     |        |
    No        Yes
     |        |
     v        v
 [Show Error]  [Booking Confirmed]
              |
              v
        Update UI / Redirect
              |
              v
             (END)
```

Connecting Frontend to Backend

1️⃣ **Set up the backend to serve the frontend**

Since your React frontend is **SPA (Vite)** and Django backend is **REST API**:

1. **Keep them decoupled** (recommended)
   - Django only exposes API endpoints (/api/...)
   - React handles all UI and routing.
2. **Enable CORS** in Django so frontend can access the backend from a different origin.

```
# settings.py
INSTALLED_APPS += ['corsheaders']

MIDDLEWARE = ['corsheaders.middleware.CorsMiddleware'] + MIDDLEWARE

CORS_ALLOWED_ORIGINS = [
  "http://localhost:5173",  # your Vite dev server
  "https://yourfrontenddomain.com"
]
```

2️⃣ Connect React frontend → Django backend

🔹 API Client (already exists)

You already have:

src/services/apiClient.js

- Configure baseURL to point to your Django backend.

Example using Axios:

```
import axios from "axios";

const apiClient = axios.create({
 baseURL: import.meta.env.VITE_API_BASE_URL || "http://localhost:8000",
 headers: {
  "Content-Type": "application/json",
 },
});

// Add JWT token automatically
```

```
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("access_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default apiClient;
```

3️⃣ Match frontend services to backend apps

Your backend apps:

Django App	React Service
authapp	auth.service.js
users	users.service.js
listings	listings.service.js
bookings	bookings.service.js

- Each React service calls the corresponding backend endpoints.
- Example: listings.service.js

```
import apiClient from "./apiClient";

export const getListings = async () => {
  const response = await apiClient.get("/api/listings/");
  return response.data;
};
```

4️⃣ Handle authentication

1. Backend: authapp provides JWT tokens.
2. Frontend: AuthContext.jsx + useAuth.js stores and provides token.
3. API client sends token in Authorization header.

Flow:

LoginPage -> useAuth() -> auth.service.login() -> apiClient.post("/api/auth/login/")

-> Response { access, refresh }

-> store in localStorage -> AuthContext.setUser()

5️⃣ Consume backend endpoints in pages

Example: Booking a property

```
PropertyDetails.jsx
  |
  v
useAuth() -> check login
  |
  v
bookings.service.createBooking(propertyId)
  |
  v
apiClient.post("/api/bookings/", { property_id })
  |
  v
Backend booking view validates -> creates booking -> returns JSON
  |
  v
Service updates frontend state / shows success or error
```

6️⃣ Align routes & URLs

- Frontend URLs: /login, /listings, /property/:id
- Backend URLs: /api/auth/login/, /api/listings/, /api/bookings/
- Always **prefix backend routes with /api/** to avoid collisions with React routes.

7️⃣ Optional: Deployment setup

- **Development**: React runs on Vite dev server, Django on localhost:8000. Use CORS.
- **Production**: Options:
    1. Serve frontend via **Django's static files** (collectstatic)
    2. Serve frontend separately (Vercel, Netlify) and point API requests to Django backend.

8️⃣ Checklist for connecting frontend ↔ backend

1. Backend exposes REST endpoints for all apps.
2. Backend allows CORS from frontend.
3. Frontend apiClient configured with backend baseURL.
4. Frontend services match backend apps.
5. Auth flow: JWT token stored, sent in headers.
6. Pages call services → services call backend → response updates state/context.
7. URLs are aligned: frontend SPA URLs vs backend /api/....